

OPAA installieren – Was Sie brauchen

OPAA (Open Project AI Assistant) läuft vollständig im eigenen Haus: als Container-Stack auf einem Linux-Server, mit lokalen Sprachmodellen und ohne Abhängigkeit von externen Cloud-Diensten. Dieser Flyer fasst zusammen, was vor der Installation bereitstehen muss.

1. Server

Komponente	Voraussetzung
Betriebssystem	Linux (64-bit, x86_64), z. B. Debian, Ubuntu, RHEL
Container-Laufzeit	Docker Engine mit Docker Compose v2
Netz	Einmaliger Zugriff auf <code>ghcr.io</code> zum Laden der Images (oder Bereitstellung über eine interne Registry)
Ports	Frontend <code>3000</code> , Backend-API <code>8081</code> — üblicherweise hinter einem Reverse-Proxy mit TLS
Speicher	Persistente Volumes für Datenbank, hochgeladene Originale und ggf. Modelle

Das Zielsystem braucht **keinen Quellcode-Checkout** und **keine Build-Werkzeuge** — die fertigen Images kommen aus der GitHub Container Registry (`ghcr.io/criew/opaa-backend`, `ghcr.io/criew/opaa-frontend`) und werden per `docker compose pull && docker compose up -d` installiert und aktualisiert.

2. Im Stack enthalten (keine separate Installation nötig)

- **PostgreSQL 18 mit pgvector** — Datenbank und Vektorspeicher in einem
- **Objektspeicher (S3-kompatibel)** für die Originalablage — optional, alternativ Dateisystem
- **Keycloak** — optional als gebündelter Identitätsanbieter
- **Ollama** — optionales Compose-Profil, nur zum Ausprobieren (siehe Abschnitt 3)

3. Sprachmodelle (Pflicht)

OPAA braucht **zwei Modelle** über eine **OpenAI-kompatible Schnittstelle** (`/v1`):

Rolle	Zweck
Chat-Modell	Antworten erzeugen — bestimmt die Antwortqualität maßgeblich
Embedding-Modell	Dokumente durchsuchbar machen — ein späterer Wechsel erzwingt eine vollständige Neuindizierung, deshalb vor dem Start festlegen

Empfohlen: ein dedizierter Modellserver mit GPU.

- **vLLM** (oder ein vergleichbarer OpenAI-kompatibler Inferenzserver) im eigenen Haus, mit einem leistungsfähigen Chat-Modell — z. B. **Gemma 4 31B**. Das ist der vorgesehene Produktivbetrieb: alle Daten bleiben im Haus, Durchsatz und Antwortqualität sind für den Mehrbenutzerbetrieb ausgelegt. Ein Modell dieser Größe braucht eine GPU mit ausreichend Speicher (Richtwert: ab ca. 24 GB VRAM bei quantisierter Ausführung, für volle Genauigkeit entsprechend mehr).
- **Gehosteter OpenAI-kompatibler Anbieter** (z. B. Azure OpenAI, ein Rechenzentrums-Dienst, LiteLLM als Zwischenschicht), wenn keine eigene GPU-Infrastruktur vorhanden ist. Zieladresse und Schlüssel werden eingetragen. *Hinweis:* Dann verlassen Anfragen das Haus; OPAA verhindert das nicht technisch — die Freigabe muss organisatorisch geklärt sein.

Nur zum Ausprobieren: Ollama im Stack (`--profile ollama`). Dieses Profil startet einen Ollama-Server auf CPU mit den Voreinstellungen `phi3:mini` (Chat) und `nomic-embed-text` (Embedding). Es eignet sich, um OPAA ohne weitere Infrastruktur kennenzulernen — **nicht für den Produktivbetrieb**: `phi3:mini` ist ein sehr kleines Modell mit entsprechend begrenzter Antwortqualität, und CPU-Inferenz ist für mehrere gleichzeitige Nutzer zu langsam.

Voreingestellt ist immer der lokale Betrieb — ohne Konfiguration ruft OPAA kein externes Modell auf.

Optional: ein **Reranking-Modell** (eigener Endpunkt) verbessert die Trefferqualität, ist aber standardmäßig abgeschaltet.

4. Anmeldung (Pflicht: eine Entscheidung)

OPAA läuft im Betrieb ausschließlich im Modus `oidc`. Darin stehen zwei Wege offen:

- **OIDC-Anbieter** (z. B. Keycloak, Entra ID, ein bestehendes SSO) — empfohlen, wenn ein Verzeichnisdienst vorhanden ist. Der gebündelte Keycloak dient als Einstieg.
- **Lokale Benutzerverwaltung** — OPAA führt die Konten selbst, ohne externen Anbieter.

Für den Erststart wird die **E-Mail-Adresse des ersten Systemverwalters** benötigt.

5. Optional, aber empfohlen

- **SMTP-Server** — für Einladungen, Passwort-Rücksetzung und Benachrichtigungen
- **Reverse-Proxy mit TLS** (nginx, Traefik, Caddy) — Zeitüberschreitungen großzügig setzen, wenn Modelle auf CPU laufen
- **Interne CA** — kann in den Truststore des Backends ergänzt werden
- **Quellen zum Indexieren** — Dateiablagen, Confluence, HTTP-Verzeichnisse, RSS, S3

Checkliste vor dem Start

- ☐ Linux-Server mit Docker und Docker Compose v2
- ☐ Zugriff auf `ghcr.io` (oder gespiegelte Images)
- ☐ Modellserver erreichbar (vLLM oder anderer OpenAI-kompatibler Anbieter), Chat- und Embedding-Modell festgelegt
- ☐ Nur für einen Testlauf: Ollama-Profil als Übergangslösung
- ☐ Anmeldeverfahren entschieden: OIDC-Anbieter oder lokale Konten
- ☐ E-Mail-Adresse des ersten Systemverwalters
- ☐ Persistente Volumes und Sicherungskonzept für Datenbank und Originale

Ausführliche Anleitung: `docs/handbuch/deployment.md` im Projekt-Repository.